LEARN BEING FORWARD

Programming — Python Programming

Control flow & functions

COMPLETE LEARNING GUIDE

What You'll Learn

Control flow determines the order code executes in, and functions are how that logic gets packaged into reusable, testable units — together they're the backbone of every Python program beyond a single script. This sub-module covers conditional branching, the for/while loop forms and their less-familiar loop-else clause, Python 3.10's structural pattern matching (match/case) as a cleaner alternative to long if/elif chains, and function mechanics in depth: default arguments, *args/**kwargs for flexible call signatures, variable scope and the global/nonlocal keywords, lambda expressions, and the basics of decorators — a pattern used constantly in real frameworks (Flask routes, pytest fixtures) that's worth understanding from first principles rather than treating as magic.

1. Conditional Branching: if / elif / else

Python's conditional statement is if, with optional elif branches for additional conditions and a final optional else for the fallback case — there is no switch statement in the classic sense (match/case, covered below, fills that role from 3.10 onward). Conditions can be chained for comparisons in a way many languages don't support: 0 <= age < 18 reads naturally and evaluates exactly as a mathematician would expect, rather than requiring two separate comparisons joined by and.

- Chained comparisons like 0 <= x < 100 are valid Python and more readable than (0 <= x) and (x < 100).
- elif is Python's spelling of 'else if' — there's no separate elseif or else if keyword.

PYTHON

```python
def classify_age(age: int) -> str:
    if age < 0:
        return "invalid"
    elif age < 13:
        return "child"
    elif age < 20:
        return "teenager"
    else:
        return "adult"

print(classify_age(15))  # teenager
```

2. Loops: for, while, range(), enumerate(), and zip()

A for loop in Python iterates directly over any iterable (a list, string, dict, or the output of range()) rather than managing an index manually, which is both more readable and less error-prone than an index-based C-style loop. range(start, stop, step) generates a lazy sequence of integers. enumerate(iterable) yields (index, value) pairs when the index is genuinely needed, and zip(a, b) pairs up corresponding elements from multiple iterables for parallel iteration — both are idiomatic replacements for manual index bookkeeping. while loops run as long as a condition holds and are the right choice when the number of iterations isn't known in advance.

- Prefer `for item in items` or `for i, item in enumerate(items)` over manually indexing with `for i in range(len(items))`.
- zip() stops at the shortest input iterable, silently dropping extra elements from longer ones.

```python
names = ["Asha", "Ben", "Cy"]
scores = [88, 92, 79]

for i, name in enumerate(names, start=1):
    print(f"{i}. {name}")

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

3. break, continue, and the Loop else Clause

break exits a loop immediately; continue skips to the next iteration without finishing the current one. A less familiar feature is the for...else / while...else clause: the else block runs only if the loop completed without hitting a break, which makes it a clean way to express 'search and act only if nothing was found,' avoiding a separate found-flag variable.

- The loop's else clause runs when the loop finishes normally (no break) — it does not run after a break.

```python
def find_first_even(numbers: list[int]) -> int | None:
    for n in numbers:
        if n % 2 == 0:
            print(f"Found even number: {n}")
            break
    else:
        print("No even number found")
        return None
    return n

find_first_even([1, 3, 5, 7])   # No even number found
find_first_even([1, 3, 4, 7])   # Found even number: 4
```

4. Structural Pattern Matching with match/case

Python 3.10 introduced match/case, which is far more powerful than a simple switch — it can match literal values, destructure sequences and dictionaries by shape, bind matched sub-values to names, and attach guard conditions with if. It's most valuable when a long elif chain is really checking the 'shape' of a value (a tuple's length, a dict's keys) rather than a single scalar — in those cases match/case reads far more clearly than nested isinstance/len checks.

- The underscore `_` is the wildcard pattern that matches anything, playing the role of a `default` case.
- Patterns can destructure directly: `case (x, y):` binds x and y from a 2-element sequence in one line.

```python
def handle_command(command: tuple) -> str:
    match command:
        case ("move", x, y):
            return f"Moving to ({x}, {y})"
        case ("stop",):
            return "Stopping"
        case ("speed", value) if value > 100:
            return "Speed too high!"
        case _:
            return "Unknown command"

print(handle_command(("move", 3, 4)))      # Moving to (3, 4)
print(handle_command(("speed", 150)))      # Speed too high!
```

5. Defining Functions: Defaults, *args, and **kwargs

Functions are defined with def, and parameters can have default values (used when the caller omits that argument), must be listed after any non-default parameters. *args collects any extra positional arguments into a tuple, and **kwargs collects extra keyword arguments into a dict — both allow a function to accept a flexible, variable-length call signature, which is how libraries build decorators and wrapper functions that forward arguments to another function without knowing its exact signature in advance.

- A mutable default argument (def f(items=[])) is a classic Python trap — the same list object is reused across every call that doesn't pass its own; use `items=None` and create the list inside the function instead.
- Keyword-only arguments can be forced by placing a bare `*` in the parameter list before them.

```python
def build_profile(name: str, *hobbies: str, **extra_info) -> dict:
    return {"name": name, "hobbies": hobbies, **extra_info}

profile = build_profile("Nia", "chess", "hiking", age=29, city="Nairobi")
print(profile)
# {'name': 'Nia', 'hobbies': ('chess', 'hiking'), 'age': 29, 'city': 'Nairobi'}
```

6. Scope, global, and nonlocal

A name assigned inside a function is local to that function by default, even if a name with the same spelling exists at module level — Python decides this statically by scanning the function body for any assignment to that name. To modify a module-level (global) variable from inside a function, the global keyword must declare the intent explicitly; to modify a variable from an enclosing (but not global) function in a nested function, nonlocal is used instead. Relying on either sparingly is good practice — passing values in and returning results out is almost always clearer than mutating shared state through scope keywords.

```python
counter = 0

def increment():
    global counter
    counter += 1

increment()
increment()
print(counter)  # 2

def make_accumulator():
    total = 0
    def add(amount):
        nonlocal total
        total += amount
        return total
    return add

acc = make_accumulator()
print(acc(5))  # 5
print(acc(3))  # 8
```

7. Lambda Functions and Higher-Order Functions

A lambda is a small, unnamed, single-expression function — lambda x, y: x + y is exactly equivalent to a def with one return statement, just without a name and limited to one expression (no statements, no multiple lines). Lambdas are most useful as a short throwaway `key` function passed to sorted(), max(), or min(), rather than as a replacement for regular function definitions, which remain more readable for anything beyond a one-liner.

- If a lambda needs a name to be referenced more than once, it should usually just be a regular `def` function instead.

```python
people = [("Ana", 32), ("Ben", 24), ("Cy", 45)]

# Sort by age (the second tuple element) using a lambda as the key
sorted_by_age = sorted(people, key=lambda person: person[1])
print(sorted_by_age)  # [('Ben', 24), ('Ana', 32), ('Cy', 45)]
```

8. Decorators: Wrapping Functions With Extra Behavior

A decorator is a function that takes another function and returns a new function that wraps it with extra behavior — logging, timing, access control — without modifying the original function's own code. The @decorator_name syntax above a function definition is pure sugar for my_func = decorator_name(my_func). Writing one from scratch (rather than just using framework-provided ones) makes clear that decorators are ordinary functions and closures underneath, nothing more exotic.

- functools.wraps should wrap the inner function to preserve the original function's __name__ and docstring — omitting it is a common, subtle decorator bug.

```python
import functools
import time

def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper

@timed
def slow_square(n: int) -> int:
    time.sleep(0.1)
    return n * n

slow_square(5)  # prints: slow_square took ~0.1000s
```

Common Pitfalls

- Using a mutable default argument (def f(items=[])) and being surprised that the same list persists and accumulates across unrelated calls.

- Writing manual index-based for loops (`for i in range(len(items))`) instead of iterating directly or using enumerate().
- Forgetting the loop's `else` clause only runs when the loop completes without a `break`, not whenever the loop 'finishes' in general.
- Omitting the `global` keyword when trying to reassign a module-level variable inside a function, creating an unrelated local variable instead.
- Writing a multi-line or multi-statement lambda attempt — lambdas are restricted to a single expression and cannot contain statements.
- Forgetting `functools.wraps` in a custom decorator, which silently replaces the wrapped function's `__name__` and docstring with the wrapper's.

Key Takeaways

- Chained comparisons (0 <= x < 100) and the for/while `else` clause are idiomatic Python features with no direct equivalent in many other languages.
- match/case (3.10+) is for matching the *shape* of data (sequences, mappings), not just a value — it's more powerful than a simple switch.
- *args and **kwargs let a function accept a flexible, variable-length signature, which is how wrapper functions and decorators forward arbitrary calls.
- Never use a mutable object as a default argument — use None and create the mutable value inside the function body instead.
- global and nonlocal are the two keywords for intentionally modifying a non-local variable; absent either, assignment always creates a new local name.
- A decorator is just a function that takes a function and returns a (usually wrapping) function — the @ syntax is sugar for manual reassignment.

Learn More

Python Docs — More Control Flow Tools
https://docs.python.org/3/tutorial/controlflow.html

Python Docs — PEP 634: Structural Pattern Matching
https://peps.python.org/pep-0634/

Real Python — Primer on Python Decorators
https://realpython.com/primer-on-python-decorators/